

Distributed Computing for Intelligent Buildings in Communities: The Caravels Approach

Rafael Silva¹[0000-0001-8260-6820], Luis Gomes¹[0000-0002-8597-3383], and Zita Vale¹[0000-0002-4560-9544]

¹ GECAD Research Group on Intelligent Engineering and Computing for Advanced Innovation and Development, LASI—Intelligent Systems Associate Laboratory, Polytechnic of Porto, R. Dr. António Bernardino de Almeida, 431, 4200-072 Porto, Portugal
{rdpds, log, zav}@isep.ipp.pt

Abstract. The transition toward decentralized energy systems requires intelligent, resilient, and user-centered infrastructures capable of supporting both autonomy and cooperation across buildings. This paper presents Caravels, which is a distributed system architecture designed to enable intelligent communities through container-based orchestration, focusing in two new features: personalized user modeling, and inter-building service and data sharing. Building upon a modular microservice deployment paradigm, Caravels integrates a graph-based user preference module for context-aware personalization and a data sharing mechanism that facilitates peer-to-peer cooperation. This paper presents a three-part case study evaluating the system across heterogeneous environments deployed in a community setting. Results demonstrate real-time adaptability through dynamic service deployment based on user-defined preferences, effective inter-building collaboration via shared IoT access and service consumption, and personalized automation through reinforcement learning embedded in graphs. The system maintains low-latency operation and minimal resource usage, validating its applicability to low-cost, edge-constrained environments. Caravels innovations enable distributed cooperation on edge, sharing data while preserving sovereignty, and context aware personalization at the building-level.

Keywords: Container Orchestration, Intelligent Buildings, User Preference Modeling.

1 Introduction

The ongoing digital transformation of energy systems, accelerated by the European Union’s objective to increase at least 32% renewable energy of the final energy consumption by 2030, states that countries should move towards decentralized renewable energy production which will require consumer involvement [1]. This objective, while motivated by environmental measures, also reflects a broader shift toward independent and resilient energy infrastructures where communities assume an active role in energy production, management, and optimization [2]. In this context,

intelligent buildings acting as producers, consumers, and prosumers, will compose intelligent communities emerging as a promising architectural model to achieve both sustainability and technology integration.

Proliferation of Internet of Things (IoT) devices, be it in residential, office or industrial settings, combined with the rise of container-based orchestration and edge computing, has enabled the deployment of distributed intelligence across diverse physical locations [3]. These developments make it feasible to implement context-aware energy management systems, capable of adapting to local conditions and individual preferences while maintaining coordination with larger-scale objectives [4]. However, the transition towards intelligent buildings and communities is not without its challenges, as the integration of such technologies also introduces challenges related to computational complexity, interoperability, data privacy, resource constraints and system scalability, particularly when relying on single-board computers (SBCs) in edge computing scenarios [5], [6].

In recent years, container orchestration frameworks have evolved to support low-footprint, distributed execution of microservices across heterogeneous devices [7]. These solutions facilitate service modularity, fault tolerance, and rapid deployment, which are essential for communities that must respond dynamically to environmental and user-driven factors. Nevertheless, two dimensions remain insufficiently addressed in the current literature and practice.

Existing approaches generally treat buildings as a monolithic entity, ignoring their role in the broader community, and intra-building heterogeneity of users, contexts, and preferences. In intelligent environments where AI models make decisions that impact comfort, cost, and sustainability, such oversimplifications not only lead to suboptimal or inequitable outcomes but also limit user acceptance and trust [8]. While distributed systems enable local intelligence, they rarely incorporate mechanisms for inter-building collaboration. The ability to share services and data securely across multiple buildings within a community could significantly enhance forecasting accuracy, resilience, especially under conditions of resource scarcity or demand volatility, and the creation of new solutions for coordinating communities [9]. Traditional intelligent community implementations rely on centralized architectures for data aggregation, this introduces single points of failure and significant privacy risks. Demoting users of sovereignty over their data.

This work is an extension of the previous work published and presented in SOCO 2024, where Caravels were first proposed. Specifically, [10] introduced the foundational Caravels container-based architecture and validated the feasibility of modular service deployment and baseline resource efficiency on edge hardware. However, it did not address user personalization or cooperative inter-building data exchange. This paper extends that foundation with two novel contributions:

- (i) a new graph-based user preference model that enables personalized service adaptation based on individual profiles, and contextual variables; and
- (ii) a new inter-building data sharing mechanism, which facilitates secure, peer-to-peer exchange of IoT data and services among households.

Together, these contributions substantially extend the capabilities of the first version of Caravels: the first version established the foundational multi-building container

orchestration layer and validated its modularity, this paper adds contextual intelligence and active inter-building cooperation, enabling buildings to personalize services to individual users and to exchange IoT data and services with peers, while preserving data sovereignty, low latency, and system resilience.

The remainder of this article is structured as follows. Section 2 presents the state of the art in intelligent communities, intelligent buildings, preference modeling, and data sharing between these buildings. Section 3 details the Caravels system architecture. Section 4 describes experimental setup and case study. Section 5 discusses the results, including system performance, adaptability, and privacy considerations. Section 6 concludes the paper and outlines directions for future research.

2 State of the art

This section will address the main topics of this work and describe developments made in the studies topics considering recent literature. The concepts which will be addressed are intelligent communities, intelligent buildings, user modeling, and inter-building data sharing.

2.1 Intelligent Communities

An energy community is a type of community-driven institution that takes social control of shared energy resources through decentralization, where individual consumers, producers, and prosumers in an enclosed topology can establish such space to build independent projects and actively contribute to a more sustainable paradigm [11]. Energy communities enable the promotion of democratization and decentralized energy production, fostering resilience to mitigate and avoiding disruptions in the grid and high energy prices [12].

There are several components that energy communities can include, such as energy generation sources, where renewable sources are prioritized contributing to reducing reliance on fossil fuels [13], energy storage systems that can store excess energy generated which can be used to increase flexibility, and management and monitoring systems providing ways to track energy consumption, generation, and distribution in real-time [14].

In the EU, there are already several energy communities developed. In [15], the results of preliminary fieldwork investigations of sustainable energy community development approaches in two Danish islands and one rural village in Ireland are reported. The authors' major evidence suggests that social barriers are interrelated and frequently reinforce each other, posing the most significant challenge to deploying energy communities. In [16], it is proposed the development of a smart energy community would bring savings of 66% relative to energy costs, as the community would have a self-consumption of nearly 95%, meaning less power would have to be transported on the network.

Regarding informatic systems, most community implementations rely on centralized architectures for data aggregation and decision-making [17]. This

introduces limitations such as a single point of failure, scalability, privacy concerns when storing on a central server, and lack of adaptability [18]. Recent literature is shifting this paradigm through decentralized and edge-based alternatives, though gaps remain. For instance, [19] utilizes local edge nodes to host services, but they remain administratively rigid and do not allow for user-driven deployment. Others, such as [20] provide protocols for inter-organizational data sharing but rely on central data silos.

2.2 Intelligent Buildings

Containerization has become the de facto paradigm for enabling lightweight, portable, and scalable deployment of applications in distributed systems [21]. This is due to the ability of containers to isolate applications and their dependencies in a single package, enabling easy deployment and consistency across different environments [22]. Orchestration frameworks such as Kubernetes and its lightweight derivatives (e.g., K3s, MicroK8s) facilitate service management across heterogeneous and resource-constrained devices [23]. These tools can offer resource limit control, optimize the placement of containers on specific nodes, distribute the load among container instances, verify if a container needs to be destroyed or replaced, create containers automatically if applications or nodes fail, and automatically scale applications.

Prior works have demonstrated the efficacy of container orchestration in edge environments, supporting low-latency operations for IoT applications ranging from controlling robots to smart warehouses [24], [25]. Container orchestration technologies can even enable hybrid computing environments where support for resource-limited at the edge and resource-rich resources at the cloud are combined [26].

In the domain of intelligent buildings, recent efforts in hybrid edge-cloud architecture indicate the potential for adaptive orchestration based on both local constraints and global optimization goals, yet comprehensive frameworks integrating personalization and inter-building cooperation remain underdeveloped [27], [28]. Nevertheless, current frameworks often treat services as static and fail to adapt dynamically to evolving user contexts or inter-building collaboration opportunities. Caravels introduces a dynamic virtual private network that allows nodes to expose specific services to peers on-demand, optimizing resource usage across the community without compromising the host infrastructure.

2.3 User Preferences Modeling in Intelligent Buildings

User modeling is a foundational challenge in intelligent building systems, where personalized comfort, energy efficiency, and user satisfaction often generate conflicts. Traditional approaches have relied on rule-based systems and simple parameterizations to define user preferences [29]. More recent work explores machine learning techniques, including Reinforcement Learning (RL) for adaptive systems control [30] and ontology-based reasoning for context-aware services [31], to capture dynamic, non-stationary user behaviors.

However, there are a number of challenges on how to structure user preferences. These can be how to create a representation of these preferences, such that users can understand what the system predictable behavior will be, or how will services interact with a given structure, so that a value can be retrieved and utilized by these services. These challenges further increase when considering the complexity and heterogeneity of the IoT devices present in the scenario, and the presence of multiple users in the same environment.

Graph-based representation has gained traction for representing complex, relational preference structures, particularly in multi-user environments where conflicts must be negotiated [32]. However, few implementations integrate graph-based user modeling with containerized, distributed infrastructures in intelligent building settings, particularly where real-time responsiveness and data privacy are critical.

By representing users, preferences, contexts, and values as interconnected nodes in a graph-based structure, Caravels allows for multi-user conflict resolution, and dynamic adaptation. This formalizes the negotiation of comfort and cost at the edge level.

2.4 Inter-Building Data Sharing

While intra-building context awareness has seen significant advancements, the potential for inter-building data sharing and collaboration remains largely untapped [33]. Greater interconnectedness between buildings would create opportunities for new applications and services, as well as provide more comprehensive and advanced capabilities, such as context, to optimize models.

The potential for collaboration among buildings, whether for load balancing, joint forecasting, or cooperative demand response, has been recognized in the smart grid and microgrid literature [34]. Learning approaches, which train models across distributed nodes without centralizing data, offer privacy-preserving frameworks well-suited to intelligent energy communities [35]. Similarly, secure multi-party computation and differential privacy techniques provide mechanisms for collaborative intelligence without compromising individual autonomy [36].

Despite these advancements, practical implementations of inter-building collaboration remain scarce, often hampered by interoperability issues, resource constraints at the edge, and the lack of lightweight, semantically aligned communication protocols. Existing proposals focus predominantly on numerical optimization without integrating richer semantic models of user intent, context, and service requirements, a gap that this work addresses.

3 System Overview: Caravels

In Caravels, a community is built by connecting existing, independent building systems, each one being a caravel. In this way, Caravels is a System of Systems (SoS), both at the community level and at the building level, giving that IoT devices compose intelligent buildings, which in turn compose an intelligent community. This architecture

grants several benefits both for independence and coordination across levels: (i) each IoT device/building operates independently but contributes data and actions to the building/community intelligence; (ii) each building system is individually managed but consider broader decision-making at the community level (iii) adding or remove services do not require entire architecture redesign; (iv) buildings are physically distributed, yet digitally interconnected.

Caravels draw its name from the Portuguese ships of the 15th and 16th centuries, which enabled maritime expansion through their robustness, adaptability, and capacity for both individual exploration and fleet coordination. Analogously, Caravels envisions a distributed computing environment for intelligent communities where individual buildings' machine (caravel) maintain autonomy while contributing to a larger collaborative ecosystem.

Each caravel encompasses all the hardware and software resources deployed within a building physical and digital space, including IoT devices, edge computing nodes, and cloud services. Individual buildings within a community each host an instance of the Caravels framework, responsible for managing local resources, interacting with users, and coordinating with other caravels.

This distributed infrastructure is orchestrated via Kubernetes, allowing both the orchestration of containers inside a building, and the orchestration of the overall community. From the container orchestrator's point-of-view, each caravel is a node and in turn the community is a cluster. Each service is packaged in the form of a container, typically it is just one, but depending on the service can be more. This means that each service can be individually scaled, updated and deployed across the caravel cluster.

As caravels do not share the same network, the system creates a Virtual Private Network, where each caravel can expose specific services. This prevents Caravels having to expose services to the public internet, introducing unnecessary risk. Generally, the exposed services refer to the ones that need to communicate with the community services, for example, to coordinate a voluntary demand-response event [37]. However, each caravel can individually decide to expose new services as it deems fit.

Each caravel will host specific services based on user personalization or hardware requirements, as shown in Fig. 1. A deployed container will be an instance of a service and users in one household will not be able to access the services of another household, unless intended.

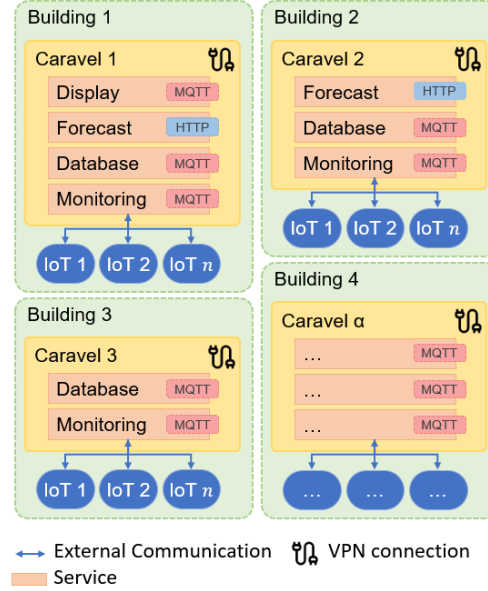


Fig. 1. Solution's Architecture

As through the virtual private network (VPN) caravels share the same network, they can communicate using peer-to-peer connections. It allows for lower latencies, which can be useful for applications that use real-time adjustments, and enables the deployment of applications, at the community level, that require collaboration or exchange of information between caravels.

3.1 User Preference Module

Traditional rule-based approaches often treat buildings as single entities, failing to account for intra-building heterogeneity and conflicting user preferences. To make a user-friendly interface that users can understand, interact and further personalize the caravel to their own preferences. A graph-based approach is used to represent each user's preferences, priorities, and contextual information, providing a transparent, digital representation of user profiles that can be dynamically merged and adapted across different contexts. It is intended that this approach can evolve and grow over time, by the user adding new needs, features and preferences. The structure needs to be capable of addressing changing temporal contexts, both in short term and long term. Each structure is linked to one user, making it a unique digital representation.

The structure has different types of nodes:

- User: represent the user of the structure, identifying who the structure belongs to.
- Preference: linked to a specified preference, such as preferred lighting or temperature.

- Condition: specifies a condition that changes the current context. For example, user location, time of day or season.
- Context: related to the contexts defined by the condition nodes.
- Value: a value for the preference it belongs to. It can have several types such as unitary, range, list or a machine learning model.

Through the combination of these nodes, the resulting structure closely resembles a tree. With a specific user as the root, it branches out into different preferences, and as the conditions of those preferences vary, other contexts are added, ending value nodes as leaf. In order to enhance the structure evolution, the system registers interactions, which will allow users to modify the value nodes, adjusting the value they hold to the user inputs. For example, if a user is increasing temperature every day at 6 PM, this indicates that temperature should be increased to the defined value at this time. A primary innovation of this module is the integration of RL model as a discrete and isolated value node. By modularizing the learning process, changes in one branch are isolated from others, preventing adaptive learning from degrading broader models' performance. Additionally, the model environment boundaries can be defined by upstream nodes, restricting the learning process to, for example, "Time of Day".

Apart from the owner of the structure, it also needs to interact with present services from the building and with structures of other users. To support different modifications and interactions with the structure a set of operations were defined:

- Planting: Addition of a new node to the structure.
- Weeding: Deletion of a given node.
- Harvesting: Extracts the current value of a specified preference.
- Trimming: Cuts a part of the overall structure.
- Grafting: Merging two structures.
- Flourishing: Interacting with the structure through direct or indirect interactions.

When inferred by a service the structure will evaluate the current context, based on the defined conditions, and harvests a value for a given preference. However, in a normal scenario, more than one person can be present in the building, and their preferences may contradict each other. To solve this conflict, the system will trim all the structures that belong to present users to the specified preference and then graft them into a new one, creating a temporary structure which the service will now harvest. In this conflict resolution process, several strategies may be implemented, such as priority, threshold-based, or weight-based strategies. These strategies are defined by the users, selecting the ones that fit the given use case, and can vary according to the services.

As users often move between buildings, the system must be capable of importing the already defined structure of the user in one building, to another one. This avoids the user having to "fresh start" the structure in a new building. Especially because users generally maintain their preferences [38].

The graph-based structure offers high expressiveness while maintaining computational efficiency suitable for SBC-based edge nodes. It represents user preferences and adapts to changing contexts, enabling intelligent building services and personalized automation scenarios.

3.2 Inter-Building Sharing

In order to achieve collaboration, caravels need to communicate with each other. This communication must be secure, and preserve the intended access policies, as each building may have different user permissions.

As already stated, caravels are not in the same local network, unless they are open to the public internet, they cannot communicate directly. To achieve this a VPN tunnel is used to connect all caravels into a single network, using end-to-end encryption [39]. Through this network, caravels can expose their given services, devices or establish direct communication. The chosen tool to enable this is Headscale [40], an open-source and self-host version of Tailscale [41].

A caravel may expose certain services, such as weather forecasting, to the network under a controlled and authenticated framework, through Headscale. For instance, a caravel with a more powerful model for weather forecasting, may expose it to other caravels, which would benefit from having more accurate forecasting in the scope of intelligent services. Services are deployed as containers, as such when exposing them, instead of exposing the entire host machine, only the container running the desired service is exposed. Meaning that access is restricted to that given container, preventing unintended access. This is achieved by combining k8s with Headscale, implementing a zero-trust infrastructure, users are isolated to only their own devices and specific shared resources. These policies are enforced on every node's incoming packet filter limiting the exposure of the infrastructure, especially the caravel in question.

Sharing IoT device data is achieved by opening a route to the given IoT device. This route can then be used by another caravel to communicate with the IoT device as it was in the same local network. This device is then registered in the given caravel as a shared device.

To guarantee the reliability and security of these communications, this module provides concrete mechanisms across four dimensions. First, all inter-caravel traffic is encrypted using WireGuard, the protocol underlying Headscale, which establishes authenticated tunnels via elliptic-curve public-key cryptography [42], where each caravel holds a unique key pair, and only nodes whose public keys are registered in the Headscale controller are admitted to the overlay network. Second, authentication is enforced at the network layer, peer relationships are explicitly provisioned in the Headscale access control lists (ACL) configuration, meaning an unrecognized caravel cannot join the mesh regardless of network reachability. Third, fine-grained access control is implemented through per-node ACL policies specifying precisely which services or IoT routes each peer may access. These rules are enforced on every node's ingress packet filter, preventing any lateral access beyond the intended scope. Fourth, containment in the event of node compromise is achieved through two complementary mechanisms: (i) Kubernetes namespace and network-policy isolation confines each service to its own container namespace, limiting a compromised container's access to host resources or co-located services, and (ii) the VPN overlay restricts the blast radius of a compromised node to only the explicit routes and service endpoints it was already authorized to reach, leaving all other caravels unreachable. Each caravel enforces

locally defined policies specifying which data or services can be shared, under which conditions, and with which peers.

3.3 Deployable containers

Each caravel does not directly provide functionalities, it relies on the services it is running. This means that the system will provide as many features as the user wants. Inherently, requirements such as compatibility, privacy, and security are passed on to the service making the application on the container ensure these requirements.

Under each service is an application that implements its functionalities. This implies that a defined service will have predetermined functionalities, with its route, parameters, and return value types already outlined. By having multiple applications that achieve the same outcome, the user can choose what application he prefers whatever those reasons may be (e.g., efficiency, computational requirements, or efficacy). To become part of the available services catalog, new applications can easily announce their capabilities and connect with other nodes in the network. It is establishing a common language for describing services and their functionalities through a standardized Application Programming Interface (API) that enables seamless integration.

The system can achieve a more efficient and scalable software deployment and management by containerizing each application. Each container runs in its own isolated space, ensuring resource isolation and enhancing security by limiting the impact of potential vulnerabilities within one container on others. It also guarantees high portability, meaning that a containerized application can run seamlessly across different computing environments minimizing dependencies on a specific host environment. As the system needs containers to measure any type of results, 4 different services were developed, an Energy Monitoring Service, a Storage Service, an Energy Forecasting Service, and a Display Service.

4 Case studies

To validate the practical applicability, modularity, and scalability of Caravels, a real-world study across a multi-node deployment simulating a real-world intelligent community is conducted. This case study is divided into three parts. The first part, as already published in [10], analyzes the modularity of services for each caravel. The second part focuses on the modeling of user preferences and the application of graph-based structures. The third part demonstrates the applicability of data sharing.

To support the transparency and reproducibility of this study, the deployment logic and orchestration configuration files used in the multi-node case study, including MicroK8s service manifests, Headscale network configuration, inter-caravel ACL policies, and a step-by-step deployment runbook for reproducing the seven-node Raspberry Pi setup, are available in a public Zenodo repository [43]. Furthermore, a comprehensive technical description of the Caravels architecture, including detailed

implementation parameters and extended preliminary testing, is documented in a Master dissertation [44].

4.1 Service Modularity

The deployment consists of seven nodes, each representing a distinct caravel within a local energy community. All nodes were deployed on Raspberry Pi 4B devices each equipped with 4 GB of RAM. Each machine is running MicroK8s for container orchestration. Each caravel had access to a subset of IoT devices, such as energy meters, light sensors, and PV sensors.

Table 1 outlines the configuration of services for each caravel. This case study implements the following core services:

- **Energy Monitoring Service:** Collects real-time data from energy meters and publishes consumption/generation readings over MQTT.
- **Storage:** Persists MQTT messages from the subscribed MQTT topics into a local database.
- **Forecasting:** Performs hourly energy consumption or generation forecasting using a pre-trained Support Vector Machine model and publishes the forecasts over MQTT.
- **Display:** Depending on the services available on the given node displays energy consumption, generation, and forecasts on E-Ink displays.

Table 1 Node configuration

Services	Nodes						
	1	2	3	4	5	6	7
Storage	✓	✓	✓	✓	✓	✓	✓
Energy Monitoring (consumption)	✓	✓	✓	✓	✓	✓	✓
Energy Monitoring (generation)	✓	✓			✓		
Energy Forecasting (consumption)	✓	✓		✓	✓	✓	✓
Energy Forecasting (generation)	✓	✓			✓		
Display	✓		✓	✓			

Fig. 2 shows the nodes used in the deployment of the proposed case study. For simplification, the representation of IoT devices connected are not shown. The MQTT communication enables the service communication, while Caravels HTTP network, enabled by Kubernetes, enables the communication among nodes and, therefore, households. All nodes can communicate with each other.

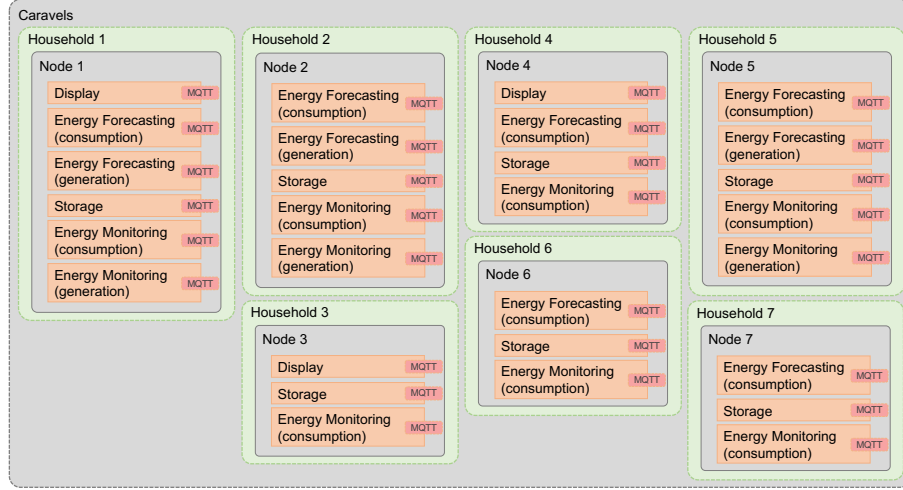


Fig. 2. Case study implementation (excluding IoT connectivity)

The Energy Monitoring service can monitor consumption in 5 second periods, and as it publishes the read data, the Storage service can subscribe to the MQTT topic and save the given data. After that, we can request the Storage service to aggregate and obtain historical data, for each node. Applying this to all the nodes we can obtain the historical energy consumption of each node throughout the day, as we can observe in Fig. 3.

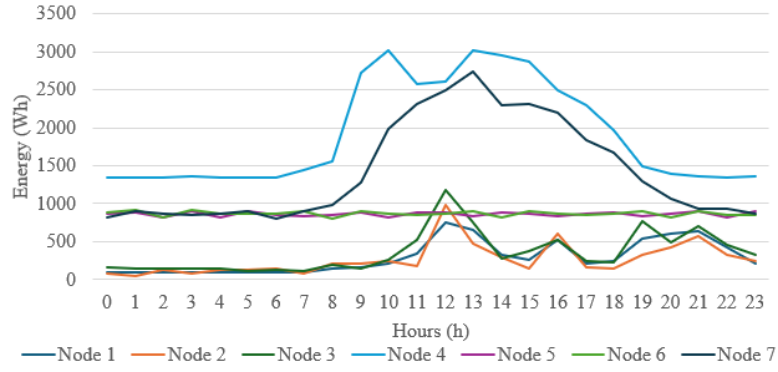


Fig. 3. Energy consumption for a 24-hour period

If the node integrates an e-ink display, as is the case of nodes 1, 3, and 4, the system can deploy a Display service that will show information based on what services are enabled on the node. For example, Fig. 4 represents the Node 3 display and because only the energy monitoring service for consumption is available, it will only display the consumption.



Fig. 4. Node 3 display

Having a dynamic interface, as provided by the display service, allows for greater adaptability without having to change the application running the Display service, as shown in Fig. 5, which represents Node 4 and Node 1, respectively.

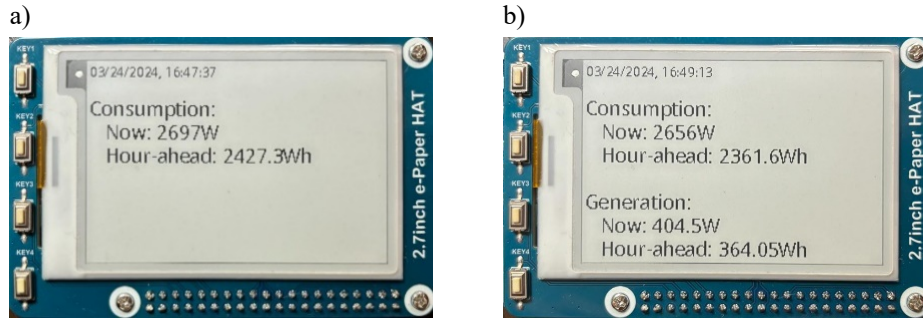


Fig. 5. e-ink display: a) Node 4, b) Node 1

4.2 User Preference Modeling

This case demonstrates the deployment and functionality of the user preference module within a single caravel. Firstly, the user manually initializes their preferences through the graphical interface, allowing him to “plant” preferences and conditions without technical knowledge. As preferences are planted in the local graph structure, they become accessible by other services of Caravels. The user sets an initial value for the value nodes, resulting in the following structure shown in Fig. 6.

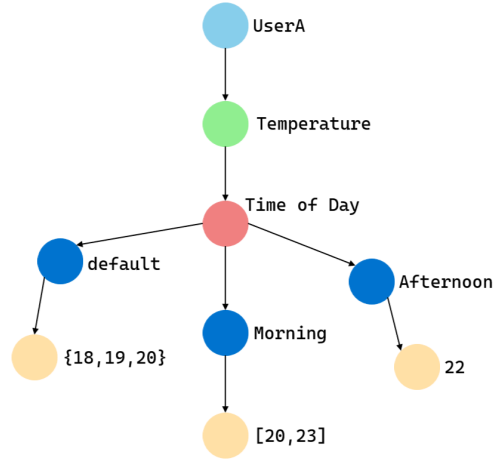


Fig. 6. Initial user's structure

Over a period of one week, the interactions with the system are recorded. The system logs the user's actions, such as adjusting the room temperature, turning lights on/off, or opening/closing window blinds, to see if user preferences are being met, and adapting the value nodes to the historical data. The type of adaptation in the value depends on the type of node. For unitary, the value node is updated averaging the current value with the new input value. For range, if the input is out of the interval, it automatically shifts the range to the left or to the right, depending on the side of the range the input value is on. To increase the size of the range the update of the range boundaries uses an exponentially weighted moving average with a smoothing factor of 0.1. For the list type the value node is appended with the new input value, and each value has a temporal decay. In Fig. 7 is shown the results of these interactions.

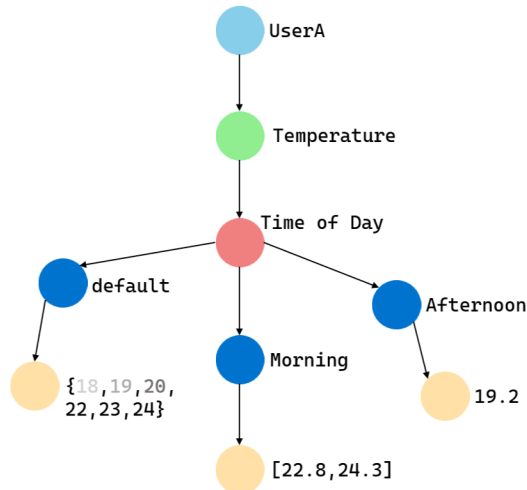


Fig. 7. User's structure after several interactions

When using the RL type, the input will be used to assign a reward, that will be the learning signal for the agent. The reward function is defined as that the agent receives a negative reward (-1) whenever the user manually overrides the system's action, and a positive reward (+1) when no override occurs. The developed agent employs the Q-learning algorithm governed by an E-greedy policy. Where the discount factor is set to 0.9, and the learning rate to 0.1. At each step, the agent selects the temperature value with the highest Q-value in the current state. Training proceeds over 1000 episodes, each comprising 20 time steps.

The agents operate by evaluating the contextual conditions defined in upstream nodes of the preference graph, such as the current Time of Day, to determine appropriate actions. However, the RL environment is extensible and supports the incorporation of additional context dimensions including environmental conditions, occupancy patterns, energy prices, and user location, thereby enabling more granular and adaptive values.

To simulate real-world mobility, User B from another caravel visits the household. The system automatically imports the user's preference structure, preserving the contextual logic. This structure is similar to User A structure shown in Fig. 7. When services harvest the preferences, they will detect the User B structure and behave according to his needs, allowing seamless transitions without user reconfiguration.

In some hours, User A and User B are present in the same room, making the system deal with multi-user conflict. Their individual structures were maintained independently and triggered during co-occupancy. When the service tries to harvest the structures, the system first trims the structures to isolate the desired preference branch, then grafts the structures to produce a merged temporary structure. This temporary structure is shown in Fig. 8, as well as the illustration of the conflict resolution. For this scenario, a dynamic weight-based strategy was chosen. This adjusts influence ratios based on occupancy time and frequency of override actions.

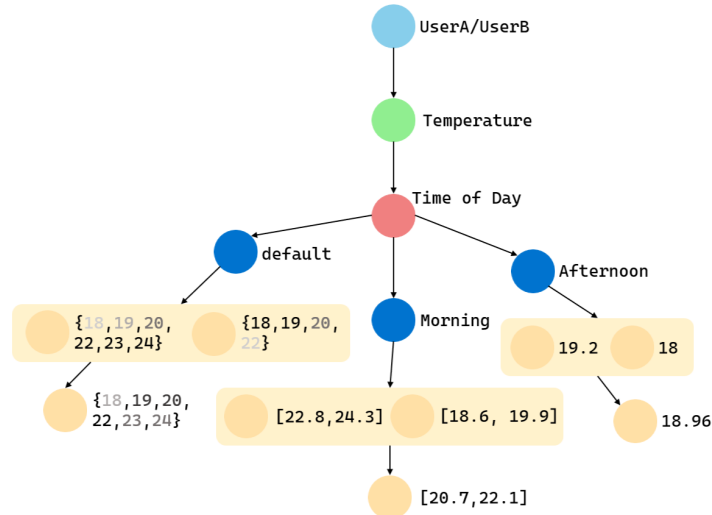


Fig. 8. Temporary structure and conflict resolution illustration

4.3 Inter-Building Data Sharing

One of the novel capabilities of Caravels is the ability to connect buildings, by supporting peer-to-peer cooperation through data and service sharing mechanisms. This section presents two scenarios demonstrating inter-building collaboration. The first involves sharing access to an IoT device, while the second involves shared consumption of a deployed service.

In the first scenario, Caravel 1 is equipped with a weather station collecting local environmental data, such as temperature, humidity, and wind speed. Caravel 2, located nearby, requests access to the sensor stream to enhance its energy forecasting capabilities.

To facilitate this, Caravel 1 opens a route through the VPN to expose the weather station. This exposes the IP address of the IoT device to Caravel 2. When Caravel 2 registers the device as a shared IoT, the system will have access to the produced data, and services present in Caravel 2 can then consume this data. Fig. 9 shows a print screen of the Caravel 2 registered IoT page.

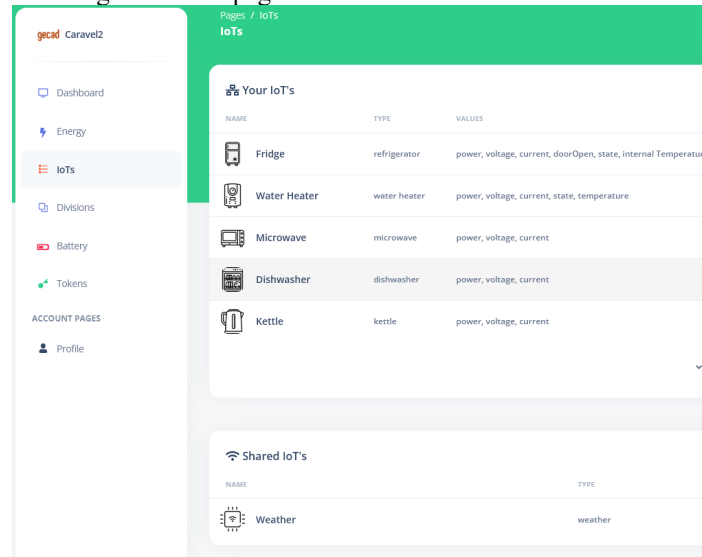


Fig. 9. Caravel2 IoT page (print screen)

In the second scenario, Caravel 2 hosts a weather forecasting service, which wraps a pre-trained model that predicts temperature, humidity and solar irradiance for the next 12 hours based on historical data. As Caravel 1 lacks the deployment resources to host such a service, it requests remote access to the weather forecast service.

Caravel 2 exposes the container where the service is deployed to the VPN, allowing Caravel 1 to access the results output by this service, as observed in Fig. 10. Forecast requests completed with a mean response time of 71.12 ms and 100% uptime during the test period. The shared service was accessed concurrently by Caravel 1 and 5 without disruption, confirming Caravels ability to support multi-tenant access and service sharing.

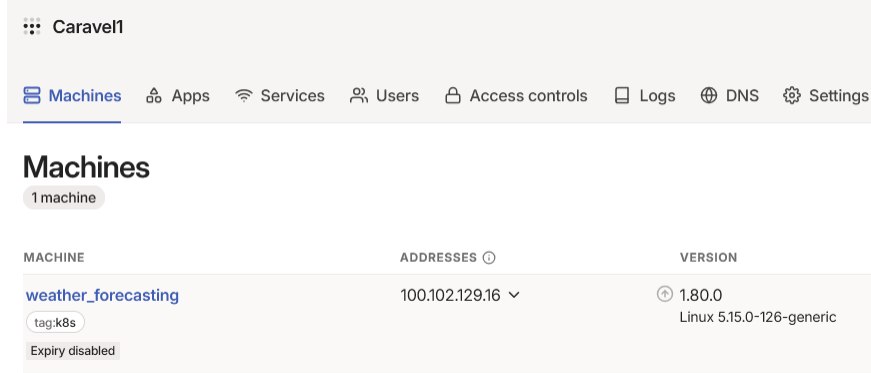


Fig. 10. Caravel1 Headscale admin console interface (print screen)

These scenarios demonstrate the practical viability of inter-building cooperation under the Caravels architecture. The ability to share data and services independently and support flexible resource sharing where caravels with constrained hardware can delegate intensive computation, high-value IoT devices can serve multiple clients, while caravels retain autonomy over their devices, services, and access policies.

5 Results and discussion

Caravels represents a paradigm shift in distributed computing for intelligent buildings and communities, leveraging edge computing principles to achieve unprecedented levels of personalization, efficiency, and resilience. The evaluation of Caravels across three distinct scenarios provides a comprehensive understanding of its capabilities in intelligent building integration and distributed computing.

To evaluate system performance on resource-constrained hardware, such as Raspberry Pi 4B, a multi-phase stress test was conducted on the cluster, comprising Caravel 1, 2, 3 and 4. Monitoring focused on CPU utilization, memory impact, and thermal stability during isolated CPU and MQTT stress periods. During the CPU stress phase, cluster utilization averaged 49.6%, representing a 1.9x increase over the baseline. Caravel 1 exhibited the highest individual load, peaking at 69.5% CPU usage, which corresponds to the higher container density on that node (9 containers versus 7–8 on other nodes). Memory usage remained stable, increasing from a baseline of 1.23 GB to 1.26 GB, a 2.1% total increase.

The MQTT stress phase achieved a cluster peak throughput of 9,345 msg/s. This volume resulted in a 9.2% increase in CPU utilization, raising the baseline from 26.1% to 35.3%. Thermal monitoring indicated a baseline temperature of 48.4°C, which reached an average of 53.2°C during stress, with an absolute peak of 65.7°C. These values remain within the thermal safety limits of the hardware. Regarding the communication infrastructure, 40,000 MQTT messages with a 256-byte payload were distributed across four brokers. The results indicated peak throughput reaching 8,986 msg/s on the most performant node, with p95 latencies remaining below 0.02 ms across all brokers. No errors were recorded during the MQTT phase, validating the reliability

of the publish-subscribe mechanism for high-frequency IoT data exchange. Additionally, 140,000 HTTP requests were issued to evaluate API endpoints, which demonstrated throughput between 239 and 265 req/s with a p50 latency of approximately 35 ms. These results confirm the architecture's capacity to maintain operations under peak community-level data loads and support high-volume service requests.

Across all scenarios, Caravels has consistently demonstrated its ability to facilitate seamless integration and performance under the constraints of single-board computers, highlighting its capability to function on resource-constrained environments. The container orchestration component maintained stable service deployment and recovery, while allowing for fast portability and configuration. However, performance and resource consumption will strongly depend on the services deployed, as such, independent benchmarking is needed to evaluate the service suitability to deployment in the host machine in question.

Caravels proved capable to adapt to each user's needs through independent deployment of services, provided by the container-based architecture, while avoiding full reconfiguration of the system.

Through structural representation of preferences, conditions, and contexts, the system enabled both individual adaptation and multi-user conflict resolution in a transparent and understandable manner. By modularizing reinforcement learning agents as discrete value nodes within the graph hierarchy, the Caravels architecture ensures that adaptive changes to a specific preference are isolated; this prevents local model updates from degrading the performance of the broader system or inducing instability in unrelated preference branches. The resolution of conflicting preferences through weight-based grafting ensured that all users had their needs addressed dynamically, balancing personalization and cohabitation. There are, however, inherent concerns regarding graph modeling that require further research; namely, determining "fair" weights becomes statistically complex as the number of concurrent users increases. Furthermore, as graphs grow in size and complexity, inconsistencies in the ordering of preference branches across heterogeneous user structures can complicate the semantic alignment necessary for seamless grafting. Moreover, the portability of preference structures highlights the potential for seamless user mobility across different Caravel-enabled environments, a significant feature for future intelligent communities where services adapt to different users independently of the building where they are located.

Inter-building data sharing showcased the capacity to create collaborative ecosystems between caravels, while guaranteeing autonomy and security. While inter-building data sharing introduces new ways of interaction, the Caravels architecture mitigates security risks through multi-layered isolation. By utilizing a VPN, the system avoids exposing services to the public internet. Furthermore, the modular nature of the microservices ensures that a compromised container does not grant access to the underlying OS or other local services. The ability to share data allows for the integration of diverse data sources and functionalities, thus leveraging the complementary strengths of each caravel. To confirm the feasibility of data sharing between caravels a latency test was performed using a containerized python script. The results from the

tests revealed an average latency of 53.04 ms and a jitter of 8.86 ms. Throughout this time, Caravel 1 retained full control over access to the IoT device and revoked access at the end of the tests without disrupting its own operations. This affirms Caravels position as a viable framework to address challenges in data and computation ownership in intelligent communities.

While this case study involved a reduced number of caravels, the modular nature of the architecture suggests good horizontal scalability, as seen in [21], making it feasible to replicate in larger communities. Nonetheless, challenges arise, such as:

- The growing complexity of maintaining and orchestrate a larger number of containers and services, across a significant number of machines.
- Conflict resolution scalability, when the number of users present in the same environment is increased.
- Guaranteeing a fair resolution of conflicts through complex preference structures, including merging embedded RL models.

6 Conclusions

This paper presented enhanced version of Caravels, a distributed system architecture which is designed to support intelligent communities through a container orchestration, graph-based user preference modeling, and inter-building cooperation. Building upon the original Caravels architecture, this paper introduces the ability to provide personalized and context aware service adaptation, and data sharing that facilitates secure exchange of data and services.

Through a three-part case study, the system demonstrated capability to support modular and resource efficient deployment across heterogeneous environments, facilitate individual and collective adaptation to user preferences through a user-friendly graph structure, and enable collaborative behavior between buildings via shared hardware and software resources. The system maintained low-latency communication and high adaptability across the scenarios, including multi-user conflict resolutions and cross-building preference portability.

By integrating containerized services, user-centered modeling, and self-hosted infrastructure, the system provides a foundational architecture for distributed AI in intelligent environments. Caravels contributes a scientifically validated framework for human-centered intelligence, proving that interconnected buildings can share resources efficiently while guaranteeing the privacy and autonomy of all participants. These attributes position Caravels as an approach that can realize the vision of intelligent communities, where interconnected buildings share resources and data to provide efficient and personalized services, while guaranteeing the privacy, security, and autonomy of all involved parts. Additionally, the system simplifies the integration of additional software and hardware, while addressing challenges in scalability, personalization, and data governance for distributed AI in intelligent environments.

As intelligent buildings evolve into autonomous, cooperative entities within the broader community, Caravels contributes with a foundational architecture for

distributed, adaptive, and human centered intelligence into the infrastructure of sustainable living.

Acknowledgments. This work has been supported by the European Union under the Next Generation EU, through a grant of the Portuguese Republic's Recovery and Resilience Plan (PRR) Partnership Agreement, within the scope of the project PRODUTECH R3 – "Agenda Mobilizadora da Fileira das Tecnologias de Produção para a Reindustrialização", Total project investment: 166.988.013,71 Euros; Total Grant: 97.111.730,27 Euros. The authors acknowledge the work facilities and equipment provided by GECAD research center (UIDB/00760/2020), DOI:10.54499/UIDB/00760/2020 to the project team.

References

- [1] "A European strategic long-term vision for a prosperous, modern, competitive and climate neutral economy," 2018. Accessed: Jan. 27, 2026. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex:52018DC0773>
- [2] I. López *et al.*, "European energy communities: Characteristics, trends, business models and legal framework," *Renewable and Sustainable Energy Reviews*, vol. 197, p. 114403, Jun. 2024, doi: 10.1016/J.RSER.2024.114403.
- [3] A. Ullah *et al.*, "Orchestration in the Cloud-to-Things compute continuum: taxonomy, survey and future directions," *Journal of Cloud Computing*, vol. 12, no. 1, pp. 135–, Dec. 2023, doi: 10.1186/S13677-023-00516-5.
- [4] S. Zeng, A. Kody, Y. Kim, K. Kim, and D. K. Molzahn, "A reinforcement learning approach to parameter selection for distributed optimal power flow," *Electric Power Systems Research*, vol. 212, p. 108546, Nov. 2022, doi: 10.1016/J.EPSR.2022.108546.
- [5] V. M. Nik and A. Moazami, "Empowering energy flexibility and climate resilience using collective intelligence based demand side management (CI-DSM)," *Journal of Physics: Conference Series*, vol. 2069, no. 1, p. 012149, Nov. 2021, doi: 10.1088/1742-6596/2069/1/012149.
- [6] Y. Himeur, A. N. Sayed, A. Alsalemi, F. Bensaali, and A. Amira, "Edge AI for Internet of Energy: Challenges and perspectives," *Internet of Things*, vol. 25, p. 101035, Apr. 2024, doi: 10.1016/J.IOT.2023.101035.
- [7] D. Yakubov and D. Hästbacka, "Comparative Analysis of Lightweight Kubernetes Distributions for Edge Computing: Performance and Resource Efficiency," in *Service-Oriented and Cloud Computing (ESOCC 2025)*, Springer, Cham, 2025, pp. 81–95. doi: 10.1007/978-3-031-84617-5_7.
- [8] Á. A. Cabrera, A. Perer, and J. I. Hong, "Improving Human-AI Collaboration With Descriptions of AI Behavior," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. CSCW1, Apr. 2023, doi: 10.1145/3579612.
- [9] F. Cirillo, D. Gómez, L. Diez, I. Elicegui Maestro, T. B. J. Gilbert, and R. Akhavan, "Smart city IoT services creation through large-scale

- collaboration,” *IEEE Internet Things Journal*, vol. 7, no. 6, pp. 5267–5275, Jun. 2020, doi: 10.1109/JIOT.2020.2978770.
- [10] R. Silva, L. Gomes, and Z. Vale, “Caravels: A Decentralized Container-Based Infrastructure for Sustainable Human-Centric Intelligent Energy Communities,” in *The 19th International Conference on Soft Computing Models in Industrial and Environmental Applications SOCO 2024 (SOCO 2024)*, Springer, Cham, 2024, pp. 241–250. doi: 10.1007/978-3-031-75013-7_23.
 - [11] V. Z. Gjorgievski, S. Cundeva, and G. E. Georghiou, “Social arrangements, technical designs and impacts of energy communities: A review,” *Renewable Energy*, vol. 169, pp. 1138–1156, May 2021, doi: 10.1016/J.RENENE.2021.01.078.
 - [12] M. A. Heldeweg and Séverine Saintier, “Renewable energy communities as ‘socio-legal institutions’: A normative frame for energy decentralization?,” *Renewable and Sustainable Energy Reviews*, vol. 119, p. 109518, Mar. 2020, doi: 10.1016/J.RSER.2019.109518.
 - [13] J. Lowitzsch, C. E. Hoicka, and F. J. van Tulder, “Renewable energy communities under the 2019 European Clean Energy Package – Governance model for the energy clusters of the future?,” *Renewable and Sustainable Energy Reviews*, vol. 122, p. 109489, Apr. 2020, doi: 10.1016/J.RSER.2019.109489.
 - [14] E. Barbour, D. Parra, Z. Awwad, and M. C. González, “Community energy storage: A smart choice for the smart grid?,” *Applied Energy*, vol. 212, pp. 489–497, Feb. 2018, doi: 10.1016/J.APENERGY.2017.12.056.
 - [15] E. Heaslip, G. J. Costello, and J. Lohan, “Assessing good-practice frameworks for the development of sustainable energy communities in Europe: Lessons from Denmark and Ireland,” *Journal of Sustainable Development of Energy, Water and Environment Systems*, vol. 4, no. 3, pp. 307–319, Sep. 2016, doi: 10.13044/J.SDEWES.2016.04.0024.
 - [16] A. Giordano, E. Ghiani, F. Pilo, and L. Rosetti, “Planning of Energy Production and Management of Energy Resources in Local Energy Communities: The Case of Berchidda Municipality (Italy),” *Proceedings 2019, Vol. 20, Page 16*, vol. 20, no. 1, p. 16, Jul. 2019, doi: 10.3390/PROCEEDINGS2019020016.
 - [17] E. Palomar, C. Cruz, I. Bravo, and A. Gardel, “Performance Evaluation of a Collaborative IoT Framework for Energy-Efficient Communities,” in *2020 IEEE 22nd International Conference on High Performance Computing and Communications, IEEE 18th International Conference on Smart City and IEEE 6th International Conference on Data Science and Systems, HPCC-SmartCity-DSS 2020*, Institute of Electrical and Electronics Engineers Inc., Dec. 2020, pp. 861–866. doi: 10.1109/HPCC-SMARTCITY-DSS50907.2020.00114.
 - [18] M. M. Orabi, O. Emam, and H. Fahmy, “Adapting security and decentralized knowledge enhancement in federated learning using blockchain technology:

- literature review,” *Journal of Big Data*, vol. 12, no. 1, pp. 55–, Dec. 2025, doi: 10.1186/S40537-025-01099-5.
- [19] Priyanka, “Decentralized Cloud Architectures for E-Governance: Leveraging Edge Computing and Blockchain to Improve Accessibility and Trust in Public Services,” *Universal Research Reports*, vol. 11, no. 2, pp. 251–263, May 2024, doi: 10.36676/URR.V11.I2.1563.
 - [20] C. Zeginis, P. Charalampidis, E. Ntallaris, A. Fragkiadakis, K. Magoutis, and S. Papadakis, “Data Space-Aligned Smart City Data Sharing with Context Brokers,” in *2025 IEEE International Smart Cities Conference (ISC2)*, Patras, Greece: Institute of Electrical and Electronics Engineers (IEEE), Dec. 2025, pp. 1–6. doi: 10.1109/ISC266238.2025.11293282.
 - [21] Y. Aldwyan, R. O. Sinnott, and G. T. Jayaputera, “Elastic deployment of container clusters across geographically distributed cloud data centers for web applications,” *Concurrency and Computation: Practice and Experience*, vol. 33, no. 21, p. e6436, Nov. 2021, doi: 10.1002/CPE.6436.
 - [22] C. Pahl and B. Lee, “Containers and clusters for edge cloud architectures-A technology review,” in *2015 3rd International Conference on Future Internet of Things and Cloud*, Rome, Italy: Institute of Electrical and Electronics Engineers Inc., Oct. 2015, pp. 379–386. doi: 10.1109/FICLOUD.2015.35.
 - [23] Z. Wang, M. Goudarzi, J. Aryal, and R. Buyya, “Container Orchestration in Edge and Fog Computing Environments for Real-Time IoT Applications,” in *Computational Intelligence and Data Analytics*, Springer, Singapore, 2023, pp. 1–21. doi: 10.1007/978-981-19-3391-2_1.
 - [24] A. Santi Seisa, S. Gajanan Satpute, and G. Nikolakopoulos, “Comparison between Docker and Kubernetes based Edge Architectures for Enabling Remote Model Predictive Control for Aerial Robots,” in *IECON 2022 – 48th Annual Conference of the IEEE Industrial Electronics Society*, Brussels, Belgium: IEEE, 2022. doi: 10.1109/IECON49645.2022.9968933.
 - [25] M. Gomes and M. L. Pardal, “Cloud vs Fog: assessment of alternative deployments for a latency-sensitive IoT application,” *Procedia Computer Science*, vol. 130, pp. 488–495, Jan. 2018, doi: 10.1016/J.PROCS.2018.04.059.
 - [26] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, “Emerging Trends, Techniques and Open Issues of Containerization: A Review,” *IEEE Access*, vol. 7, pp. 152443–152472, 2019, doi: 10.1109/ACCESS.2019.2945930.
 - [27] O. Boiko, A. Komin, R. Malekian, and P. Davidsson, “Edge-Cloud Architectures for Hybrid Energy Management Systems: A Comprehensive Review,” *IEEE Sensors Journal*, vol. 24, no. 10, pp. 15748–15772, May 2024, doi: 10.1109/JSEN.2024.3382390.
 - [28] E. Psomakelis, A. Makris, K. Tserpes, and M. Pateraki, “A lightweight storage framework for edge computing infrastructures/EdgePersist,” *Software Impacts*, vol. 17, p. 100549, Sep. 2023, doi: 10.1016/J.SIMPA.2023.100549.

- [29] T. Yang, A. Bandyopadhyay, Z. O'Neill, J. Wen, and B. Dong, "From occupants to occupants: A review of the occupant information understanding for building HVAC occupant-centric control," *Building Simulation*, vol. 15, no. 6, pp. 913–932, Jun. 2022, doi: 10.1007/S12273-021-0861-0.
- [30] X. Ding, A. Cerpa, and W. Du, "Exploring Deep Reinforcement Learning for Holistic Smart Building Control," *ACM Transactions on Sensor Networks*, vol. 20, no. 3, May 2024, doi: 10.1145/3656043.
- [31] H. E. Degha, F. Z. Laallam, and B. Said, "Intelligent context-awareness system for energy efficiency in smart building based on ontology," *Sustainable Computing: Informatics and Systems*, vol. 21, pp. 212–233, Mar. 2019, doi: 10.1016/J.SUSCOM.2019.01.013.
- [32] M. Skocaj, P. E. I. Rivera, R. Verdone, and M. Erol-Kantarci, "Uplink Scheduling in Federated Learning: An Importance-Aware Approach via Graph Representation Learning," in *2023 IEEE International Conference on Communications Workshops: Sustainable Communications for Renaissance (ICC Workshops)*, Rome, Italy: Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1014–1019. doi: 10.1109/ICCWORKSHOPS57953.2023.10283576.
- [33] R. Eini, L. Linkous, N. Zohrabi, and S. Abdelwahed, "Smart building management system: Performance specifications and design requirements," *Journal of Building Engineering*, vol. 39, p. 102222, Jul. 2021, doi: 10.1016/J.JOBE.2021.102222.
- [34] J. Wu, H. Yi, M. Liu, and Y. Wang, "Day-ahead Optimal Scheduling for Cluster Smart Buildings Considering Power Sharing," in *IOP Conference Series: Earth and Environmental Science*, IOP Publishing, Jun. 2021, p. 012036. doi: 10.1088/1755-1315/766/1/012036.
- [35] A. R. Singh, R. S. Kumar, M. Bajaj, C. B. Khadse, and I. Zaitsev, "Machine learning-based energy management and power forecasting in grid-connected microgrids with multiple distributed energy sources," *Scientific Reports* 2024 14:1, vol. 14, no. 1, pp. 19207–, Aug. 2024, doi: 10.1038/s41598-024-70336-3.
- [36] S. Dai, F. Meng, Q. Wang, and X. Chen, "DP2-NILM: A distributed and privacy-preserving framework for non-intrusive load monitoring," *Renewable and Sustainable Energy Reviews*, vol. 191, p. 114091, Mar. 2024, doi: 10.1016/J.RSER.2023.114091.
- [37] R. Barreto, L. Gomes, and Z. Vale, "Demand Response-based Model for an Energy Community Considering Members and Electric Vehicles Participation," in *2023 IEEE International Conference on Environment and Electrical Engineering and 2023 IEEE Industrial and Commercial Power Systems Europe (EEEIC / I&CPS Europe)*, Madrid, Spain: Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/EEEIC/ICPSEUROPE57605.2023.10194613.
- [38] W. El Gammal, N. Sherief, and W. Abdelmoez, "User-based Adaptive Software Development for Gamified Systems," in *ACM International*

- Conference Proceeding Series*, Association for Computing Machinery, Apr. 2020, pp. 115–122. doi: 10.1145/3397056.3397088.
- [39] “Tailscale: How it works.” Accessed: Jan. 27, 2026. [Online]. Available: <https://tailscale.com/blog/how-tailscale-works#bonus-acls-and-security-policies>
 - [40] “Headscale.” Accessed: Jan. 27, 2026. [Online]. Available: <https://headscale.net/stable/>
 - [41] “Tailscale.” Accessed: Jan. 27, 2026. [Online]. Available: <https://tailscale.com/>
 - [42] H. Jumakhan and A. Mirzaeinia, “Wireguard: An Efficient Solution for Securing IoT Device Connectivity,” Feb. 2024, Accessed: Apr. 07, 2026. [Online]. Available: <https://arxiv.org/abs/2402.02093v1>
 - [43] Silva, R., Gomes, L., & Vale, Z. (2026). Distributed Computing for Intelligent Buildings in Communities: The Caravels Approach (0.2). Zenodo. <https://doi.org/10.5281/zenodo.19555465>.
 - [44] Rafael Silva. (2025). Distributed Intelligent Management of Citizen Communities. <http://hdl.handle.net/10400.22/30449>